

Resolución: Encontrar el Voto Mayoritario en Algoritmia

Este documento presenta el diseño de un algoritmo de **Divide y Vencerás** (inspirado en el esquema de *MergeSort*) para identificar la papeleta del candidato ganador (el que tiene estrictamente más de $n/2$ votos) en un censo de n ciudadanos, utilizando únicamente la función de comparación elemental `esCoincidente` con un coste de $O(n \log n)$ llamadas.

a) Representación de Soluciones y Restricciones

1. Representación de las Soluciones

Disponemos de un conjunto de n papeletas electrónicas representadas como un vector $V = [p_1, p_2, \dots, p_n]$. No conocemos la identidad de los candidatos ni cuántos hay en total. La solución esperada es el índice i ($1 \leq i \leq n$) de cualquier papeleta que pertenezca al candidato ganador.

2. Restricciones Explícitas

La única forma de inspeccionar e interactuar con las papeletas es mediante la función proporcionada:

$$\text{esCoincidente}(p_A, p_B) \in \{\text{Verdadero}, \text{Falso}\}$$

Esta función devuelve `Verdadero` si ambas papeletas contienen un voto para el mismo candidato, o `Falso` en caso contrario.

3. Restricciones Implícitas

Garantizamos por el enunciado que existe exactamente un candidato mayoritario que ha recibido estrictamente más de $n/2$ votos en el vector total.

b) Idea y Diseño del Algoritmo (Estrategia "Merge")

Para resolver este problema en tiempo $O(n \log n)$, aplicamos una estrategia de **Divide y Vencerás**:

- División (Divide):** Dividimos el vector actual de papeletas $V[\text{izq} \dots \text{der}]$ por la mitad en dos subvectores: el izquierdo $V[\text{izq} \dots \text{mid}]$ y el derecho $V[\text{mid} + 1 \dots \text{der}]$.
- Conquista (Conquer):** Buscamos recursivamente el candidato mayoritario en cada una de las dos mitades. Sea C_{izq} el candidato mayoritario de la mitad izquierda y C_{der} el de la mitad derecha. (Si una mitad no tiene un candidato con mayoría absoluta interna, devolverá -1).
- Combinación (Merge/Combine):** Esta es la parte clave. Una papeleta solo puede ser la mayoría absoluta de un bloque grande si es la mayoría absoluta de, al menos, uno de sus dos subbloques constituyentes. Por tanto, tenemos tres escenarios de combinación:

- Si $C_{izq} = C_{der}$: Entonces ese candidato es el claro ganador mayoritario del bloque combinado actual.
- Si $C_{izq} \neq C_{der}$ (o si uno de ellos es -1): Debemos verificar cuál de los dos candidatos $(C_{izq}$ o $C_{der})$ es el verdadero ganador mayoritario del intervalo combinado actual $V[izq \dots der]$.
- **Verificación:** Contamos individualmente cuántas veces aparece C_{izq} y/o C_{der} en todo el segmento actual $V[izq \dots der]$ usando la función `esCoincidente`. Si alguno de los dos supera el umbral de la mitad del tamaño actual del segmento, se convierte en el ganador de dicho segmento. En caso contrario, el segmento actual no tiene mayoría y devuelve -1 .

c) Algoritmo en Pseudocódigo

```
// Función que cuenta cuántas veces coincide una papeleta candidata en un rango c
Funcion ContarVotos(V, izq, der, papeleta_candidata)
Inicio
    contador <- 0
    Para i <- izq Hasta der Con Paso 1 Hacer
        Si esCoincidente(V[i], papeleta_candidata) Entonces
            contador <- contador + 1
        FinSi
    FinPara
    Retornar contador
FinFuncion

// Función recursiva de Divide y Vencerás
Funcion EncontrarGanadorRecursivo(V, izq, der)
Inicio
    // Caso Base 1: El subproblema se ha reducido a una sola papeleta
    Si izq = der Entonces
        Retornar V[izq] // Una única papeleta siempre es mayoría absoluta de sí misma
    FinSi

    // Calcular el punto medio para la división
    mid <- ParteEntera((izq + der) / 2)

    // Conquistar: Resolver de forma recursiva ambas mitades
    c_izq <- EncontrarGanadorRecursivo(V, izq, mid)
    c_der <- EncontrarGanadorRecursivo(V, mid + 1, der)

    // Combinar (Merge):
    // Si ambas mitades coinciden en el mismo candidato
    Si c_izq != -1 y c_der != -1 y esCoincidente(c_izq, c_der) Entonces
        Retornar c_izq
    FinSi

    // Si difieren, verificar candidatos en todo el rango actual [izq, der]
    longitud_actual <- der - izq + 1
    mitad_actual <- longitud_actual / 2

    Si c_izq != -1 Entonces
        votos_izq <- ContarVotos(V, izq, der, c_izq)
        Si votos_izq > mitad_actual Entonces
            Retornar c_izq
```

```

        FinSi
    FinSi

    Si c_der != -1 Entonces
        votos_der <- ContarVotos(V, izq, der, c_der)
        Si votos_der > mitad_actual Entonces
            Retornar c_der
        FinSi
    FinSi

    // Si ninguno obtiene mayoría absoluta en este segmento combinado
    Retornar -1
FinFuncion

Funcion ObtenerGanadorElecciones(V, n)
Inicio
    ganador <- EncontrarGanadorRecursivo(V, 1, n)
    Retornar ganador
FinFuncion

```

d) Análisis de Coste

Complejidad Temporal (Llamadas a `esCoincidente`)

La cantidad de llamadas a la función `esCoincidente` en un vector de tamaño m se define mediante la siguiente relación de recurrencia para el caso peor:

$$T(m) = 2T\left(\frac{m}{2}\right) + f(m)$$

Donde $f(m)$ es el número de llamadas a `esCoincidente` realizadas durante la etapa de combinación (el "Merge") para un subproblema de tamaño m :

1. Comprobar si c_{izq} y c_{der} coinciden requiere 1 llamada a `esCoincidente`.
2. Contar las ocurrencias de c_{izq} en el intervalo de tamaño m requiere m llamadas.
3. Contar las ocurrencias de c_{der} en el intervalo de tamaño m requiere m llamadas.

Por lo tanto, la función de combinación tiene un coste de $f(m) \leq 2m + 1$ llamadas. Esto nos da la relación de recurrencia exacta:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n) \implies T(n) \in O(n \log n)$$

Complejidad Espacial

La profundidad de la pila recursiva es de $O(\log n)$ debido a la bisección del espacio de búsqueda en cada paso. Puesto que no se realizan copias de los subvectores de datos (se trabaja directamente por índices sobre el vector original V).